Close five gaps the Hardware unit skips

The Hardware unit already names I²C, SPI and UART, mentions the Serial Plotter, shows the capacitor symbol, defines a datasheet and warns about lithium batteries. It never teaches how any of these work. Nine beginner topics are missing: how the three serial buses work on the Uno R3's pins, sending readings as CSV, the address/data/control buses with MAR and MDR, what the clock does, Harvard vs von Neumann, hexadecimal, capacitors, a walkthrough of a real datasheet, and e-waste. The research notes give sourced content for all nine, and most of it ties straight back to the existing light-indicator circuit. The GL5528 and Kingbright WP7113ID datasheets show the circuit sits far inside every limit (about 9 mA against a 30 mA absolute maximum for the LED, about 0.6 mW against 100 mW for the LDR). The BH1750 light sensor's I²C address 0x23 makes a natural first hex example. The paper CPU in A5 can be re-traced with buses and MAR/MDR without changing its program. Two caveats matter before teaching. Much of the computer-architecture wording comes from exam-revision sites, because the Cambridge 0478 syllabus PDF could not be parsed. Several chip figures (BME280 voltages, ATmega328P "1 MIPS per MHz", the capacitor 3-digit code) come only from secondary sources or search snippets. Throughout this report, **[secondary]** marks a fact from a non-manufacturer, non-standards source, and **[unverified]** marks a claim that no fetched source confirmed. A consolidated checklist for one reviewer sits at the end.

Three serial buses explain the "I²C, SPI, UART" row the unit leaves undefined

The unit currently lists "I²C, SPI, UART" as interface options in B5 and says digital sensors can send "data over a bus" in L6. L4 already covers the 9600-baud mismatch, the "port busy" error and the warning that wires on pins 0/1 break uploads. None of that needs repeating. What is missing is how each bus works and which Uno pins it uses.

UART is a two-wire, two-device link with no clock. Asynchronous serial transfers data "without support from an external clock signal", so both ends must agree on a baud rate (bits per second) and a frame format beforehand (SparkFun: Serial Communication). A frame is one start bit, the data bits, an optional parity bit and one or more stop bits, and the common setup is "9600 8N1": 8 data bits, no parity, 1 stop bit (SparkFun). Arduino's `Serial.begin()` defaults to `SERIAL_8N1` (Arduino reference source: Serial.begin()). The start and stop bits mean **10 bits go over the wire for every 8-bit byte** (SparkFun: I2C). From that, 9600 baud carries about 960 bytes per second. This is arithmetic, and it is a good exercise. The wiring rule is to "cross the RX and TX lines" and the topology rule is "one serial bus, two serial devices" (SparkFun). The unit's existing pins-0/1 warning has a reason students should hear. The Uno has a single hardware UART, and its RX pin is also wired to the USB-to-serial converter, so a second device on that pin causes contention (SparkFun). That converter is an ATmega16U2 on the R3 (Wevolver) **[secondary]**. Arduino also warns never to connect these pins to an RS-232 port, which runs at ±12 V (Arduino reference source: Serial). The Uno R4 removes the clash: its pins 0/1 belong to `Serial1`, separate from USB `Serial` (Arduino reference source: Serial). The usual example modules, a NEO-6M GPS at 9600 baud and an HC-05 Bluetooth module with a 3.3 V RX pin, are **[unverified]**. Check the vendor datasheets before using them. The SoftwareSerial workaround on other pins is also **[unverified]**, because its reference page was not fetched.

I²C puts many addressed devices on the same two wires. The bus has two signals, SDA (data) and SCL (clock). Its drivers are open-drain, so they can only pull a line low, and pull-up resistors restore it high. SparkFun suggests starting at 4.7 kΩ (SparkFun: I2C). On the Uno R3 the pins are **A4 (SDA) and A5 (SCL)**. The

Wire library "uses 7 bit addresses throughout", reserves addresses 0–7, buffers 32 bytes per transmission, and needs pull-ups on SDA/SCL. Arduino lists onboard pull-ups only for the Mega (Arduino reference source: Wire). `Wire.setClock()` accepts 100 kHz (standard mode) and 400 kHz (fast mode) (Arduino reference source: setClock). The bus suits "short runs of wire (2-3m)" (SparkFun: I2C). The 7-bit address space allows 112 usable devices, but SparkFun's practical ceiling is about seven (SparkFun). Concrete addresses make the idea real. A **BH1750 light sensor is 0x23 with its ADDR pin low and 0x5C with it high** (RIOT-OS driver docs; Linux bh1750 driver). These are **[secondary]**: driver code, not the ROHM datasheet, which timed out, and an older RIOT commit had the mapping reversed. An Adafruit BME280 breakout is 0x77 by default and 0x76 with SDO jumpered to GND (Adafruit BME280 guide). The SSD1306 OLED's usual 0x3C (sometimes 0x3D) is **[unverified]**. Because the addresses differ, all three can share A4/A5. Students must also learn that a sketch using I²C loses A4/A5 as analog inputs. That follows from the shared pins, but no source states it **[unverified]**. The BH1750's voltage range, lux range and 5 V-tolerance on GY-302 modules are also **[unverified]**.

SPI trades extra wires for speed. It uses SCK, a controller-to-peripheral line and a peripheral-to-controller line, plus an **active-low chip-select (CS) per device** (SparkFun: SPI). On the Uno R3 those are **10 (CS), 11 (COPI), 12 (CIPO), 13 (SCK)**, also on the ICSP header, and the library is used through `#include <SPI.h>` and `SPI.beginTransaction(SPISettings(speed, MSBFIRST, SPI_MODE0))` (Arduino reference source: SPI; SPISettings). The new signal names conflict between sources. Arduino says COPI/CIPO, while SparkFun says PICO/POCI for the same lines (SparkFun: SPI). The course should pick one and also show MOSI/MISO, which older modules print. On the Uno the SPI clock divides 16 MHz to between **8 MHz and 125 kHz**, and runs should stay "up to a few feet" (SparkFun: SPI). The same BME280 breakout can speak SPI through its SCK/SDI/SDO/CS pins (Adafruit), which nicely shows that the bus is a choice, not a property of the sensor. Several SPI points still need checking. Pin 13 doubling as the onboard LED is an inference. The ATmega328P rule that pin 10 must stay an OUTPUT is **[unverified]**. The SD-card module specifics (3.3 V card, CS on pin 4 or 10, FAT formatting) are **[unverified]**.

	UART	**I²C**	**SPI**
Signal wires (+ GND)	2, TX/RX crossed	2, SDA/SCL shared	3 shared + 1 CS per device
Clock line	None; agree on baud	SCL	SCK
Devices	Exactly 2	Addressed, ~7 practical	One CS each
Typical speed on Uno	9600–115200 baud	100 / 400 kHz	125 kHz–8 MHz
Uno R3 pins	D0 RX, D1 TX (shared with USB)	A4, A5 (+ SDA/SCL header)	D10–D13 + ICSP
Arduino library	`Serial`	`Wire`	`SPI`

Sources as cited in the paragraphs above. The UART "full duplex" vs I²C "half duplex" contrast follows from the wire structure, and no fetched source states it.

B5 already covers 3.3 V **boards** (Pi, ESP32). What it lacks is the far more common beginner case: a **3.3 V breakout module on a 5 V Uno**. The bare BME280 chip runs at 1.71–3.6 V and is not 5 V-tolerant (Elektor) **[secondary]**. Adafruit's breakout adds a 3–5 V regulator and level shifting so its pins are "3-5V logic level safe" (Adafruit). For I²C, SparkFun describes tying the pull-ups to the lower voltage as a trick that works only

sometimes, and still recommends a logic-level converter (SparkFun: I2C). The teachable checklist is: does VIN have a regulator, are the data pins level-shifted, and which rail do the pull-ups go to? Cheap clones may omit the shifting. The ATmega328P input-high threshold (≈3.0 V at 5 V) is **[unverified]**.

CSV over serial is the smallest possible protocol

The sketches already `Serial.println()` one value per line. Extending that to `Serial.print(a); Serial.print(','); Serial.println(b);` gives a format that works three ways. It is a live Serial Plotter graph with one trace per comma-separated column, optionally labelled `name:value` (Elektor; Norwegian Creations). That format rule is **[secondary]**, because the official Arduino plotter page did not render. It is a Python input via pyserial, where `serial.Serial('COM3', 9600, timeout=1)` and `readline()` return one record up to `\n`, and a timeout is needed or `readline()` can block forever (pyserial docs). And it is a spreadsheet, once the lines are saved as `.csv`. One real quirk deserves a sentence. `println()` ends lines with `\r\n`, and the plotter once parsed the `\r` as an extra value (arduino-serial-plotter-webapp PR #16). Python code should therefore `.strip()` each line. Two practical claims are **[unverified]**. Opening the port resets an Uno R3 through DTR, so a script should wait about 2 s. And RFC 4180's CRLF line ending happens to match `println()`. The pedagogical payoff is the bridge to the Networks unit. A newline frames each record, a comma delimits fields, and an optional header line names them. These are the same framing and format ideas as packets.

Buses, MAR/MDR and the clock finish the paper CPU's story

A5 already teaches the PC, IR/CIR, CU, ALU and ACC, the three-step cycle, the paper CPU, the observation that "instructions and data sit in the same memory", and a clock-speed misconception. A6 already gives the ATmega328P's 16 MHz clock and memory sizes. What A5 never says is *how* an address reaches memory and how data comes back. That needs the three buses and two more registers.

Each bus is defined by what it carries and which way it goes. The address bus carries a location, one way, CPU → memory/I/O. The data bus carries data or instructions both ways. The control bus carries read, write, clock, interrupt and reset signals, and Cambridge-aligned notes call it two-way (Save My Exams: Von Neumann Architecture) **[secondary, snippet only]**. A-level notes add that individual control lines have their own direction, which is consistent with that (TutorChase AQA 16.1.4) **[secondary]**. A manufacturer source agrees on the roles. Toshiba defines the control bus as "a signal line that specifies whether to read or write", but it states no directions (Toshiba e-learning). Width is the link to A3's existing 2^n rule. **A 16-bit address bus reaches 2^{16} = 65,536 locations, and each extra line doubles that** (Cambridge Engineering). The claim that "data bus width = word size" has no beginner source **[unverified]**, so teach it carefully or leave it out. Revision sites agree on one exam habit: name what each bus carries *and* its direction (RevisionDojo) **[secondary]**.

MAR and MDR sit at the CPU's boundary with memory. The MAR holds the address being read or written, and the MDR holds the data or instruction just read or about to be written. In the fetch step, the PC is copied to the MAR, "the instruction stored at the location held by the MAR is copied to the MDR", and the PC is incremented (Seneca, Cambridge IGCSE) **[secondary]**. The MAR's contents travel on the address bus with a read signal on the control bus, and the instruction returns on the data bus (Save My Exams FDE) **[secondary, snippet only]**. Applied to the unit's own paper CPU, without changing its program, the trace becomes concrete. To fetch `LOAD 4`, PC 0 goes to the MAR and onto the address bus, the CU signals *read*, and `LOAD 4` comes back on the data bus into the MDR and then the IR. Executing it puts 4 in the MAR, signals *read*, and brings the value 7 back via the MDR into the ACC. `ADD 5` repeats that read for address 5 before the ALU adds. `STORE 6` reverses the data direction: 6 goes to the MAR, the ACC's 12 goes to the MDR and out on the data

bus, and the CU signals *write*. That last step shows why the data bus must be two-way. The STORE sequence is an inference from the definitions, because no beginner source walks a write cycle step by step **[unverified]**. Exam boards differ in naming, so the existing "IR, sometimes CIR" note should stay. Exact Cambridge 0478, OCR J277 and AQA 8525 wording on buses and MAR/MDR is **[unverified]**, since none of the specifications was read.

The clock is the metronome, and the AVR nearly keeps pace with it. Clock speed is cycles per second: 1 MHz = 10^6 and 1 GHz = 10^9 (Emory CS255 notes). Revision notes list the clock among the control-bus signals (TutorChase) **[secondary]**. Microchip's AVR core text says the CPU is "driven by the CPU clock clkCPU" and that a two-register ALU operation completes in a single clock cycle (Microchip online docs, Instruction Execution Timing). That page belongs to the ATmega324PB datasheet. The core text is shared across megaAVR parts but is **not the ATmega328P datasheet itself**. The ATmega328P's "throughputs close to 1MIPS per MHz" and "up to 20 MIPS at 20MHz" (datasheet mirror, UNL) came through search snippets, because the PDF did not parse **[secondary, check against Microchip DS40002061]**. At the Uno's 16 MHz, one tick lasts 62.5 ns and the ceiling is *about* 16 million simple instructions per second. Some instructions take two or more cycles, but their exact counts from the AVR Instruction Set Manual are **[unverified]**. This complements, rather than repeats, A5's existing "GHz isn't everything" note.

The Arduino is Harvard; the paper CPU is von Neumann. Microchip states that "the AVR uses a Harvard architecture – with separate memories and buses for program and data", and that while one instruction executes "the next instruction is pre-fetched from the program memory" (Microchip online docs) (same shared-core caveat). The cited parallel-fetch behaviour is the real reason for "about one instruction per tick". A5 says "instructions and data sit in the same memory" without naming the design. One added paragraph can name it von Neumann and contrast it with the Uno, whose program lives in flash and variables in SRAM, each on its own bus. Wikipedia calls the ATmega328 *modified* Harvard (Wikipedia: ATmega328) **[secondary]**, because program memory can still be read as data. That is footnote material. The practical consequence that Arduino constants go in flash with `PROGMEM` is **[unverified]** in these notes.

Hexadecimal: four bits per digit, and the unit already needs it

Hex is base 16 with digits 0–9 and A–F (A = 10 … F = 15). Arduino writes hex constants with a leading `0x`, accepts upper- or lower-case letters, and treats unprefixed numbers as decimal. Its worked example is `0x101` = 257. Current Arduino binary constants use `0b`, while older sketches use `B`, a known source of confusion (Arduino: Integer Constants). The "why hex?" exam answer is about people: fewer digits and fewer copying errors (Save My Exams: Hexadecimal) **[secondary]**. Standard uses are a 12-digit MAC address (48 bits), a 6-digit HTML colour (24 bits, e.g. #66FF33), `%`-encoded URL characters (Save My Exams: Uses of Hexadecimal), and error codes (Save My Exams exam questions), all **[secondary]**. One hex digit is exactly a nibble, so A3's 4-bit bit-card activity extends directly to hex. A byte is always two digits (0x00–0xFF = 0–255). The worked conversions are arithmetic and should be double-checked: 1011 0110 = 0xB6 = 182; 200 = 0xC8 = 1100 1000; **0x23 = 0010 0011 = 35, not 23**. The last one lets the BH1750 address from the I^2C section serve as the hex lesson's opening example. A 16-bit address bus spans 0x0000–0xFFFF, which links hex back to the buses. Misconceptions worth teaching are teacher guidance derived from definitions, not cited research: "the computer stores hex", "A is a letter", and "8 lines give addresses 1–256". No examiner report was retrieved **[unverified]**.

Capacitors and two real datasheets turn "check the datasheet" into a skill

The unit already gives the capacitor schematic symbol, says electrolytics are polarised (L5, B4), defines a datasheet, and separates recommended conditions from absolute maximums (B5). It never explains what a capacitor does or walks through a datasheet line by line.

A capacitor stores charge, and its rated voltage is a hard limit. Capacitance is measured in farads, with practical values in pF, nF and µF. Exceeding the voltage rating "will usually destroy" the part (SparkFun: Capacitors). Ceramics are small, cheap, near-ideal, non-polarised and "well suited to decoupling". Aluminium electrolytics (1 µF–1 mF) are polarised, with the stripe marking −, and reverse-biased they "fail spectacularly (making a pop and bursting open)" (SparkFun). The **3-digit code** on ceramics gives two significant figures plus a count of zeros, in picofarads. So "104" = 100,000 pF = 100 nF = 0.1 µF, not 104 pF, and J/K/M mean ±5/10/20 % (Industrial Monitor Direct; Zbotic). This is **[secondary]**: two vendor blogs agree, but no KEMET or Vishay page was fetched. It parallels L3's resistor colour code well.

Decoupling and bulk capacitors do different jobs. Manufacturers converge on a **0.1 µF ceramic as close as possible to each power pin, plus a larger bulk capacitor nearby**. TI's THS4281 asks for 6.8–22 µF within an inch and 0.1 µF within 0.1 inch (TI THS4281). TI's ADS1262 asks for 0.1 µF plus 1 µF (TI ADS1262). Microchip asks for 0.1 µF per power pin plus bulk per domain (Microchip: Decoupling Capacitors). Students do not need to add decoupling for the Uno's own chip. The claim that the board already carries it is reasonable but **[unverified]**, because the schematic was not checked. Bulk capacitance matters the moment B9's servo appears. A starting servo can sag the supply enough to reset the Arduino, and Adafruit's fix is "a high value capacitor (470uF or greater) between GND and 5V", longer (+) lead to 5 V (Adafruit Lesson 14). The "about 100 µF per servo" rule of thumb comes from a PDF that was found but not opened (Adafruit servo shield PDF) **[unverified]**.

A capacitor on A0 is optional for this circuit, and it does not replace software smoothing. The AVR ADC "is optimized for analog signals with an output impedance of approximately 10 kΩ or less" (Microchip: Analog Input Circuitry). That is a generic AVR page, not the ATmega328P datasheet **[secondary within Microchip]**. The divider's source resistance is R_LDR ∥ 10 kΩ, which is always under 10 kΩ, so the circuit already complies. A 100 nF ceramic from A0 to GND is harmless and gives $\tau = RC = 10\ k\Omega \times 100\ nF = 1\ ms$. That makes it a good first τ calculation. Its cutoff of about 160–265 Hz sits *above* 100/120 Hz lamp flicker, though, so B7's averaging is still what handles flicker. A 100 µF part would make readings sluggish ($\tau \approx 1$ s). This is the notes' own analysis from the cited figures. No Arduino or Adafruit page recommending the capacitor was found. One search summary claimed the capacitor "mainly adds charging time", which is wrong in practice and should not be used. Capacitor discharge safety rests only on a vendor blog (LCSC) **[secondary]**. At 5 V the risk is negligible, so teach "never open mains or flash equipment" rather than discharge procedures.

Walking through the GL5528 and the Kingbright WP7113ID

Both datasheets are short and follow the standard order: description, dimensions, absolute maximum ratings, electrical/optical characteristics with test conditions, and graphs. Neither has a separate "recommended operating conditions" table, which is itself worth teaching. Simple parts often omit it, while ICs such as the ATmega328P include one.

Datasheet line	GL5528 LDR (Rapid/TruOpto)	WP7113ID red LED (Kingbright)	This circuit	Margin
Voltage limit	150 V max (dark)	5 V reverse	≤5 V across LDR	≪ limit

Datasheet line	GL5528 LDR (Rapid/TruOpto)	WP7113ID red LED (Kingbright)	This circuit	Margin
Power limit	100 mW at 25 °C	75 mW	LDR ≤0.63 mW; LED ≈18 mW	<1 %; ~¼
Current limit	—	30 mA DC; 160 mA peak (1/10 duty, 0.1 ms)	≈8–9 mA	~⅓ of max
Characteristic	10–20 kΩ at 10 lux; ≥1 MΩ dark; γ = 0.6	VF 1.9 V typ / 2.3 V max at 10 mA; 25/50 mcd min/typ; 30°	—	—
Timing / temp	20 ms rise, 30 ms decay; −30 to +70 °C	−40 to +85 °C; derating curve	—	—

"This circuit" and "Margin" are arithmetic on the datasheet values, not quoted figures.

The table supports four teaching points. First, **design to min/max, not typ**. With 330 Ω the LED draws (5 − 1.9)/330 ≈ 9.4 mA at typical VF and (5 − 2.3)/330 ≈ 8.2 mA at maximum VF. Both sit near the 10 mA test condition at which the brightness figures apply. The LDR's 2:1 part-to-part spread at 10 lux is the datasheet's own justification for B7's per-board calibration. Second, **graphs refine headline numbers**. The 30 mA rating holds only up to about 25 °C, and the derating curve falls to about 5 mA near 85 °C. Third, **gamma explains the reading swing**. γ = 0.6 means ten times more light divides resistance by about 4, so 100 lux gives roughly 2.5–5 kΩ. That is consistent with B6's 2 kΩ "bright room" value. Fourth, **a reversed LED sits at its 5 V reverse limit**, so don't leave it that way. Kingbright also warns that excess current or temperature "may result in severe light degradation or premature failure" and says not to bend the leads "more than twice" (Kingbright).

Five provenance flags apply. The GL5528 sheet is a **distributor (Rapid/TruOpto) datasheet**, because the original Senba sheet was not found online. The course's B6 cites yet another GL55 copy. The sheet's voltage, power, temperature and wavelength units are implied by column headers, not printed in the cells. Gamma **conflicts**: 0.6 on the datasheet against 0.7 on tinkered.ai **[secondary]**, so use 0.6. The Kingbright sheet's date "01/08/2026" is ambiguous between day/month and month/day. Finally, B5's BOM lists the LED as "20 mA max", which comes from SparkFun's generic red LED. The WP7113ID's absolute maximum is 30 mA. These are different parts, not a contradiction, but the BOM should name the exact part bought and quote *its* datasheet, which is the lesson itself. SparkFun's "How to Read a Datasheet" tutorial returned 404 and survives only as a snippet (SparkFun) **[secondary]**, so the walkthrough should rest on the two real datasheets above.

A record 62 Mt of e-waste makes reuse the first lab rule

B4 covers lithium battery *fire* risk but nothing about disposal or the waste the unit's parts become. The scale is sobering. The Global E-waste Monitor 2024 reports **62 million tonnes generated in 2022, up 82 % since 2010, heading for 82 Mt by 2030**. Only **22.3 % was documented as properly collected and recycled**, and generation is rising **five times faster** than documented recycling (UNITAR press release). The same release values the embedded metals at **US$91 billion**, of which US$62 billion went unaccounted for. It notes that small equipment was 20.4 Mt with only 12 % recycled, and it defines e-waste as "any discarded product with a plug or battery" (UNITAR). These figures come from the co-publisher's press release. The full report (ITU) was not opened, and the release gives the 2030 increase as both "32 %" and "33 %". Whether a newer Monitor exists as of October 2026 is **[unverified]**.

RoHS links sustainability back to the datasheet. EU Directive 2011/65/EU restricts ten substances, including lead, cadmium and mercury. Four phthalates were added by Delegated Directive (EU) 2015/863 (European Commission). The concentration limits (0.1 % by weight, 0.01 % for cadmium) are not on that page **[unverified]**. The unit's own parts make the point vividly. The Kingbright LED is marked "RoHS compliant" and recommends lead-free SAC 305 solder (Kingbright). The GL5528's drawing labels its sensing layer **CdS, cadmium sulphide** (Rapid/TruOpto), so the LDR contains a RoHS-restricted substance. The course should therefore say that LDRs go to e-waste collection, never the bin, and should be reused. The current status of the EU RoHS exemption for CdS photocells is **[unverified]**, so state it cautiously or omit it.

Disposal and buying habits follow from the sources. The US EPA says never to put lithium-ion or other rechargeable batteries "in household garbage or recycling bins", to bag them or tape their terminals, and to take them to battery recyclers or take-back points (US EPA). That guidance is US-specific, so the lab should cite its local authority instead. UK/EU WEEE rules were not researched. Reuse is realistic here because the datasheet table shows both parts running far below their ratings. The headline practices are a labelled parts-return tray, breadboard builds instead of soldering, and dead boards and cables sent to an e-waste point. Buying from authorised distributors avoids counterfeits, which are often refurbished or discontinued parts sold as new (Octopart; Cofactr). That source is **[secondary]**, and so is its "67 % of counterfeits are discontinued parts" figure. The SIA whitepaper was not opened. A counterfeit part also makes the datasheet section meaningless, which ties the two new topics together.

One reviewer with the PDFs open can clear most of the flags

The table below collects every flagged item, grouped by the primary document that would settle it. Items marked "do not teach yet" have no source at all in the notes.

Claim	Current status	Primary source to check
ATmega328P "~1 MIPS per MHz", "20 MIPS at 20 MHz", Harvard quote, per-instruction cycle counts, 0–20 MHz at 4.5–5.5 V, VIH ≈ 0.6 VCC, pin 10 OUTPUT for SPI	Snippets or shared-core page	Microchip ATmega328P datasheet; AVR Instruction Set Manual
ADC "≤10 kΩ source" figure	Generic AVR page	Same datasheet, ADC chapter
Bus directions, MAR/MDR, CIR, hex learning objectives	Revision sites only	Cambridge 0478 2026–28 syllabus; OCR J277; AQA 8525
STORE (write) bus sequence; "data bus = word size"	Inference / no source	Textbook or syllabus
BH1750 0x23/0x5C, voltage and lux range	Driver code / do not teach yet	ROHM BH1750FVI datasheet
BME280 bare-chip voltages, bus speed limits	Secondary	Bosch BME280 datasheet
SSD1306 0x3C/0x3D; NEO-6M and HC-05 defaults; SD card 3.3 V and CS pin; SoftwareSerial limits	Do not teach yet	Vendor datasheets; Arduino SD and SoftwareSerial references

Claim	Current status	Primary source to check
Serial Plotter format; DTR reset on port open	Secondary / do not teach yet	Arduino IDE 2 Serial Plotter docs; Uno R3 schematic
ATmega16U2 role; Uno onboard decoupling; no onboard I²C pull-ups	Secondary / inference	Arduino Uno R3 datasheet and schematic
Capacitor 3-digit code; discharge safety; 100 µF per servo	Secondary / unopened PDF	A KEMET/Vishay marking guide; Adafruit servo shield guide
GL5528 units and original values	Distributor sheet	Senba original, if obtainable
RoHS limits; CdS exemption; local battery/WEEE rules; newer e-waste Monitor	Do not teach yet	EU RoHS Annex II/III text; local authority; ITU

Learner-facing docs.arduino.cc URLs were not fetched directly. The Arduino facts above cite the `arduino/reference-en` source that generates those pages, so the course can link the friendlier URL once someone confirms it renders the same text.

Conclusion

These gaps are not nine unrelated topics. They are one thread through the existing project. The BH1750's 0x23 is the hex example, and it is also an I²C device that could replace the LDR. The bus trace runs on the paper CPU the unit already has. The LDR's datasheet explains both why calibration exists and why the part is e-waste. Writing the new sections as extensions of B5–B11 and A5, not as freestanding chapters, keeps the unit's single worked example doing the teaching and avoids re-explaining what learners have already met.

The bigger risk is provenance, not accuracy. Very little in the notes is likely to be wrong, but much of the computer-architecture content rests on revision sites that paraphrase a syllabus nobody could open. Several of the most "teachable" numbers (module addresses, plotter format, the 1-MIPS-per-MHz claim) rest on snippets. The Microchip datasheet and the Cambridge syllabus alone would settle roughly half the flags. Doing that check before drafting, rather than after, would let the new sections enter review as "Drafted" with clean citations instead of inheriting the "secondary source" markers the unit is already trying to retire.